领域驱动设计入门

刘 伟

2018年9月

目 录

- 基本概念
- 团队基因
- 领域要素
- 举个例子
- 总结思考

2004年Eric Evans 发表

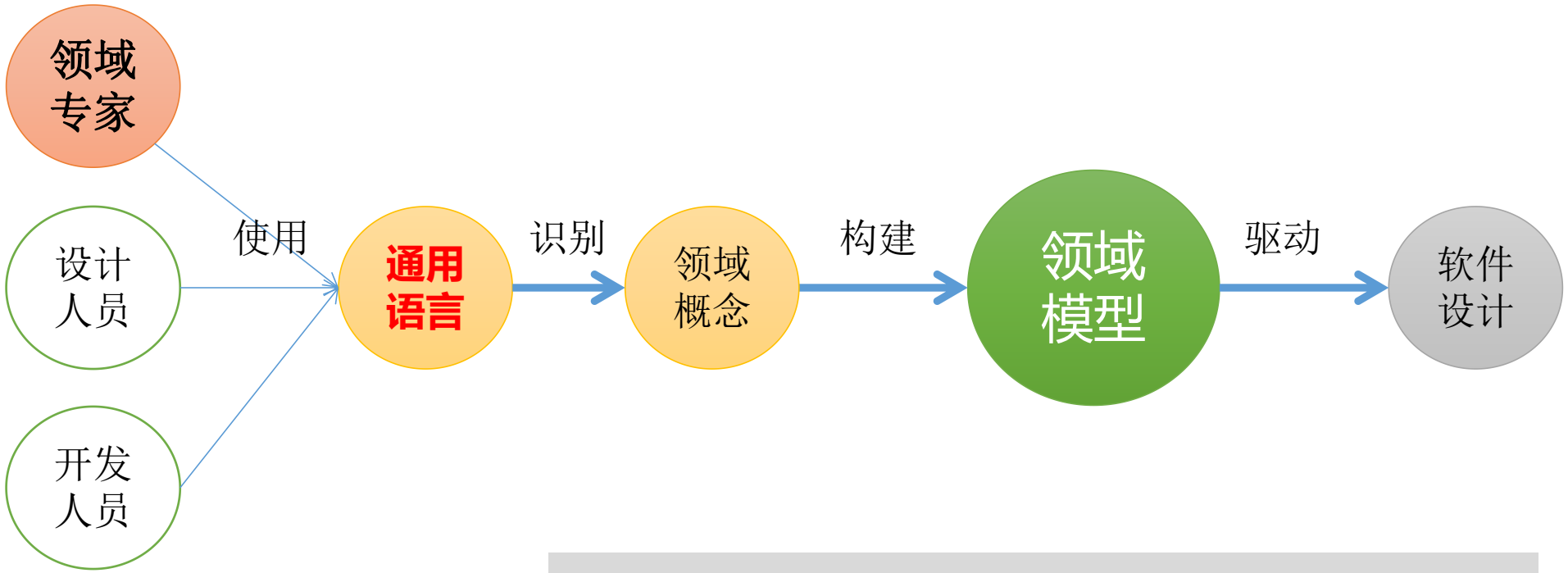
Domain-Driven Design –Tackling Complexity in the Heart of Software

简称Evans DDD，领域驱动设计。

以一种领域专家（行业专家）、设计人员、开发人员都能理解的**通用语言**作为相互交流的工具，在交流的过程中发现领域概念，然后把这些概念设计成一个**领域模型。**

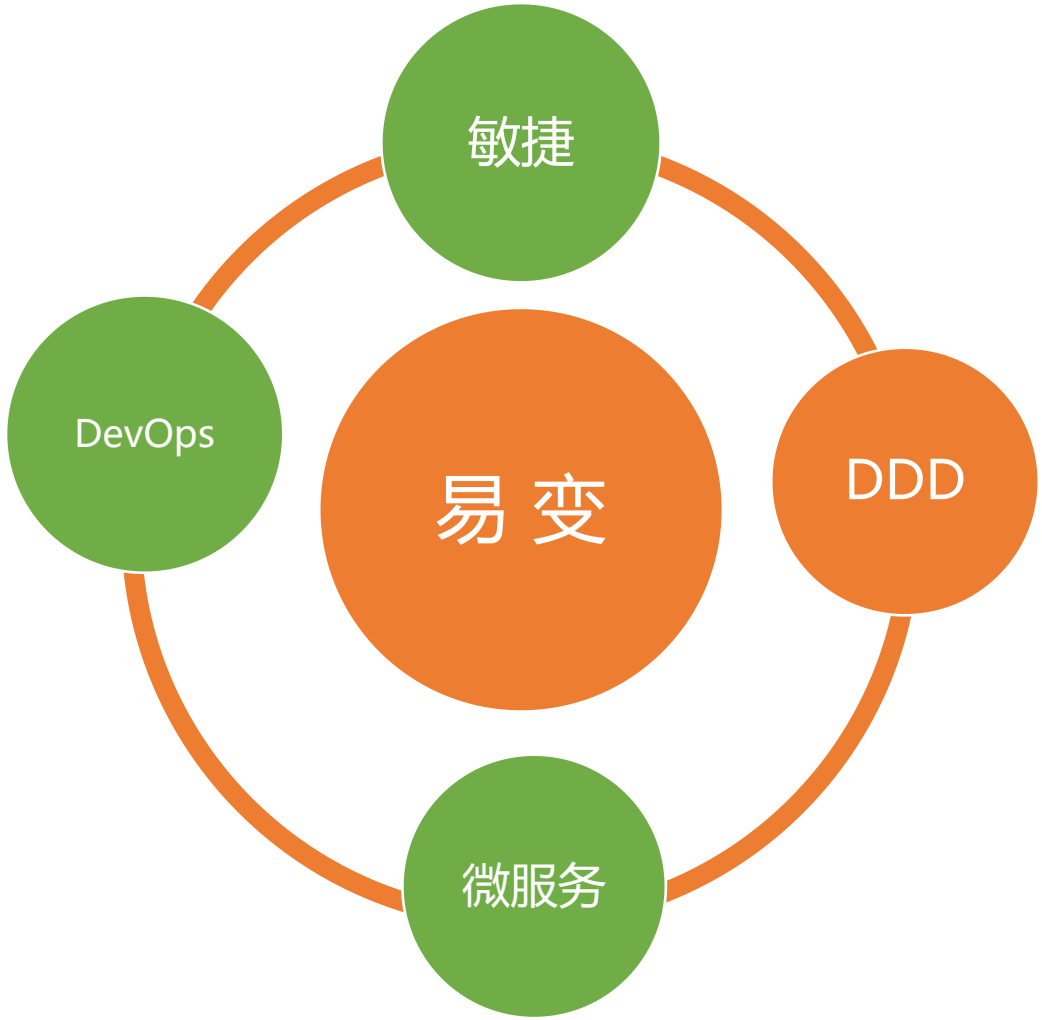
由领域模型驱动软件设计，用代码来实现该领域模型

什么是领域驱动设计（DDD）



Domain-Driven Design
--Tackling Complexity in the Heart of Software

领域模型质量影响产品业务拓展性和灵活性



领域模型是企业应用软件的灵魂和基因。

相对于各种架构模式和技术手段，领域模型离问题域更近，高质量的领域模型对提高产品适应业务变化的能力更有效。

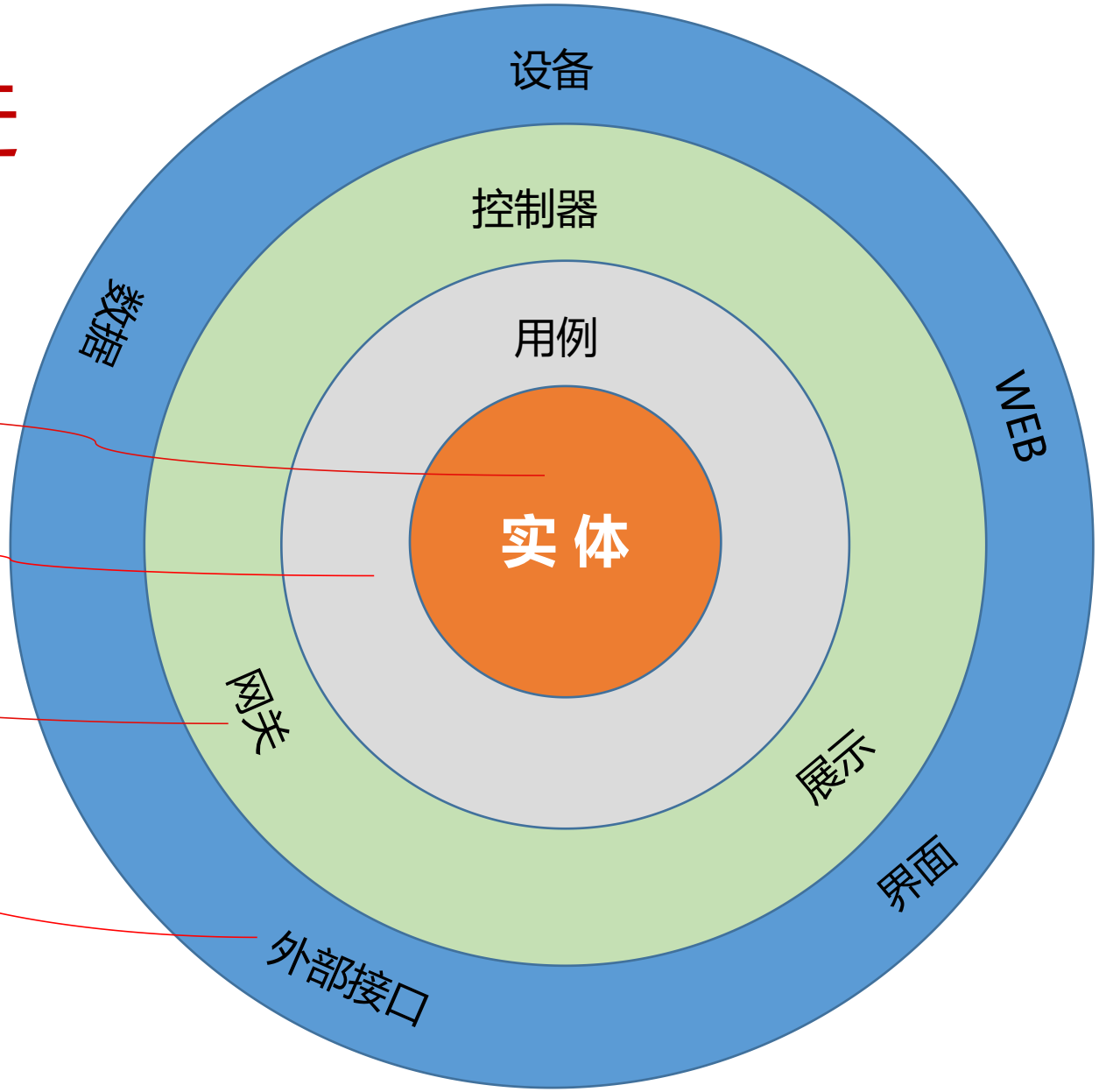
领域是稳定的，
实现方式要与时俱进

企业商业规则

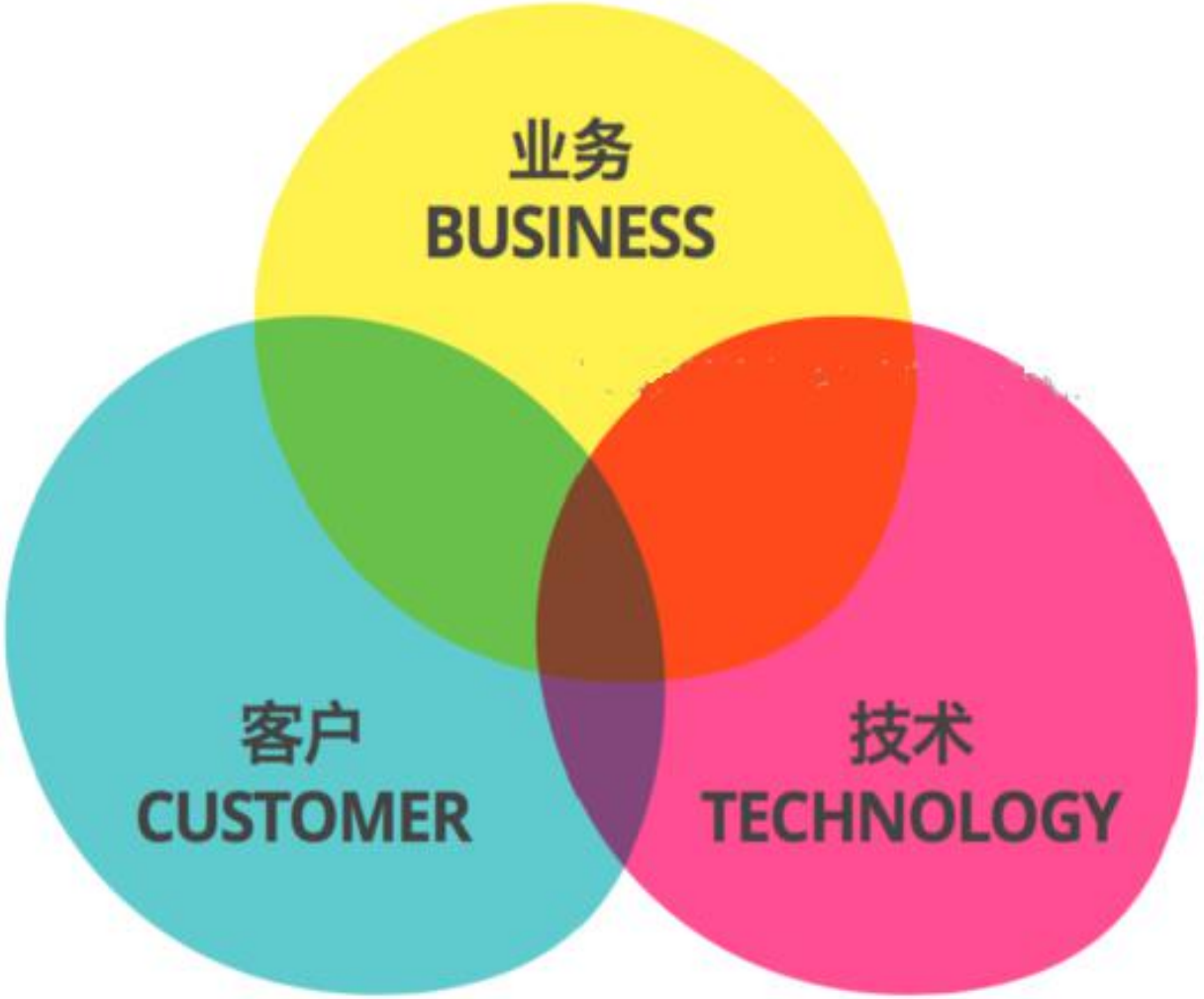
应用商业规则

接口适配

框架和驱动



DDD的核心是统一语言



领域通用语言（UBIQUITOUS LANGUAGE）

软件专家和领域专家都理解一致的语言，不存在二义性。

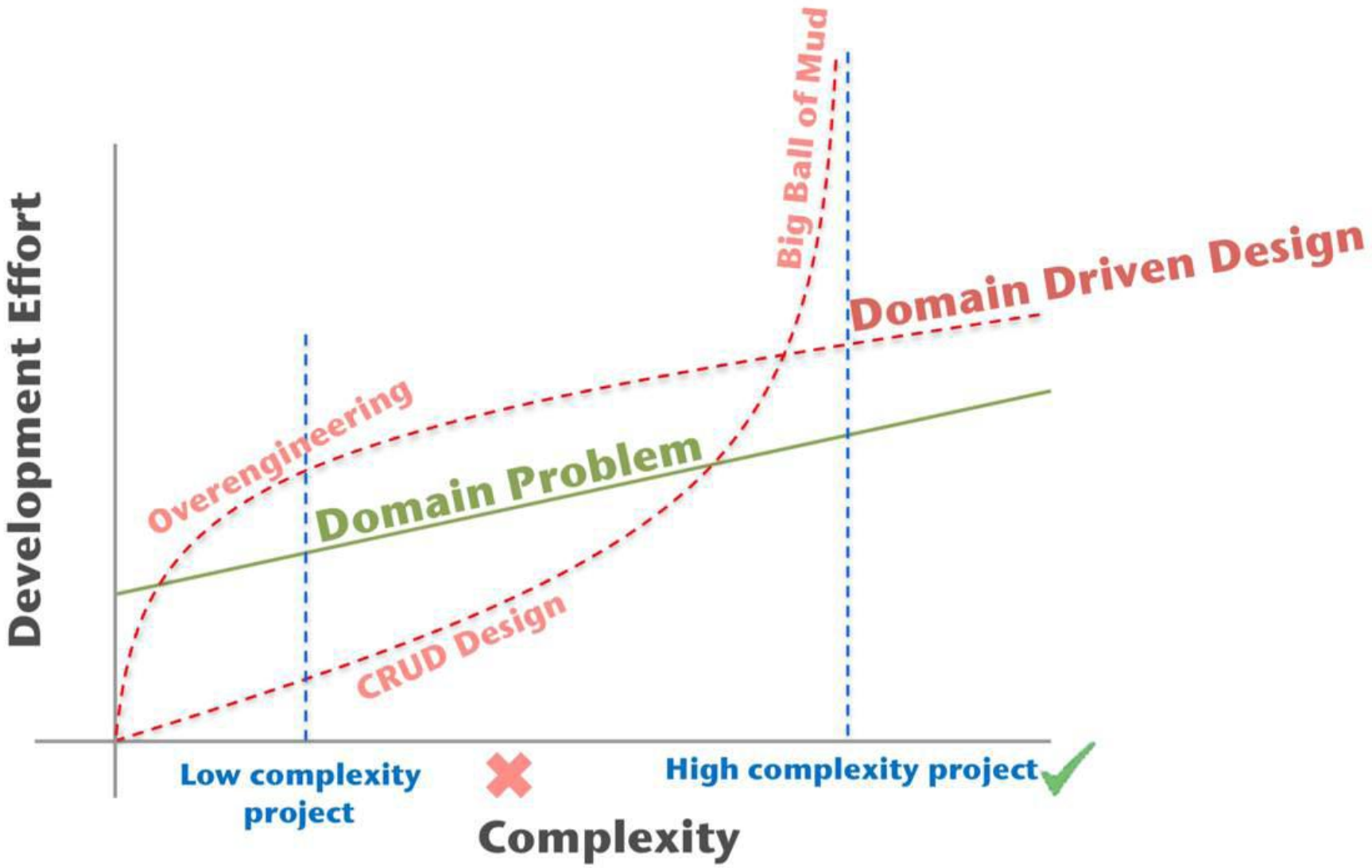
推动软件专家和领域专家的沟通，从而发现模型中的主要领域概念。

团队会使用演讲、文字、图形推敲模型和共享知识，这儿需要确保团队使用的语言在所有的交流形式中看上去都是一致的。

领域模型实施的要求

业务和系统架构绑定！

领域驱动设计优势


Big Ball of Mud
Domain Driven Design
Overengineering
Domain Problem
CRUD Design
Development Effort
Low complexity project
High complexity project
Complexity

目 录

- 基本概念
- 团队基因
- 领域要素
- 举个例子
- 总结思考

团队的基因

技术
驱动

VS

业务
驱动

开发人员

类、方法、算法、模式、架构、技术框架

生活概念 <=> 程序工件

对象类之间的关系建模

封装、继承、多态思维方式

VS

领域专家

只了解领域专业技能，

如，航空领域专家知道飞机、路线、海拔、经度、纬度，知道飞机偏离了正常路线，知道飞机的发射。

数据驱动

- 数据库优先
- 算法和数据机械结合
- 技术导向
- 代码不能反映业务
- 业务逻辑分散
- 扩展性差

VS

领域驱动

- 领域模型优先
- 算法和数据有机结合
- 业务导向
- 代码即是设计
- 业务逻辑内聚
- 扩展性佳

贫血对象

```
public class Person {
    private String name;
    private int age;
    private String phoneNumber;
    public String getName() {return name;}
    public void setName(String name) {this.name = name;}
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
    public String getPhoneNumber() {return phoneNumber;}
    public void setPhoneNumber(String phoneNumber ) {
                this.phoneNumber = phoneNumber ;
    }
}
```

上帝对象

```
public class Order {
    private Item items;
    private Date placedDate;
    addItem();
    removeItem();

    private double price;
    private int discount;
    pay();
    discount();
    issueInvoice();

    private String address;
    private String phoneNumber;
    private String postCode;
    private Status currentStatus;
    getCurrentStatus();
    changeAddress();
}
```

选购

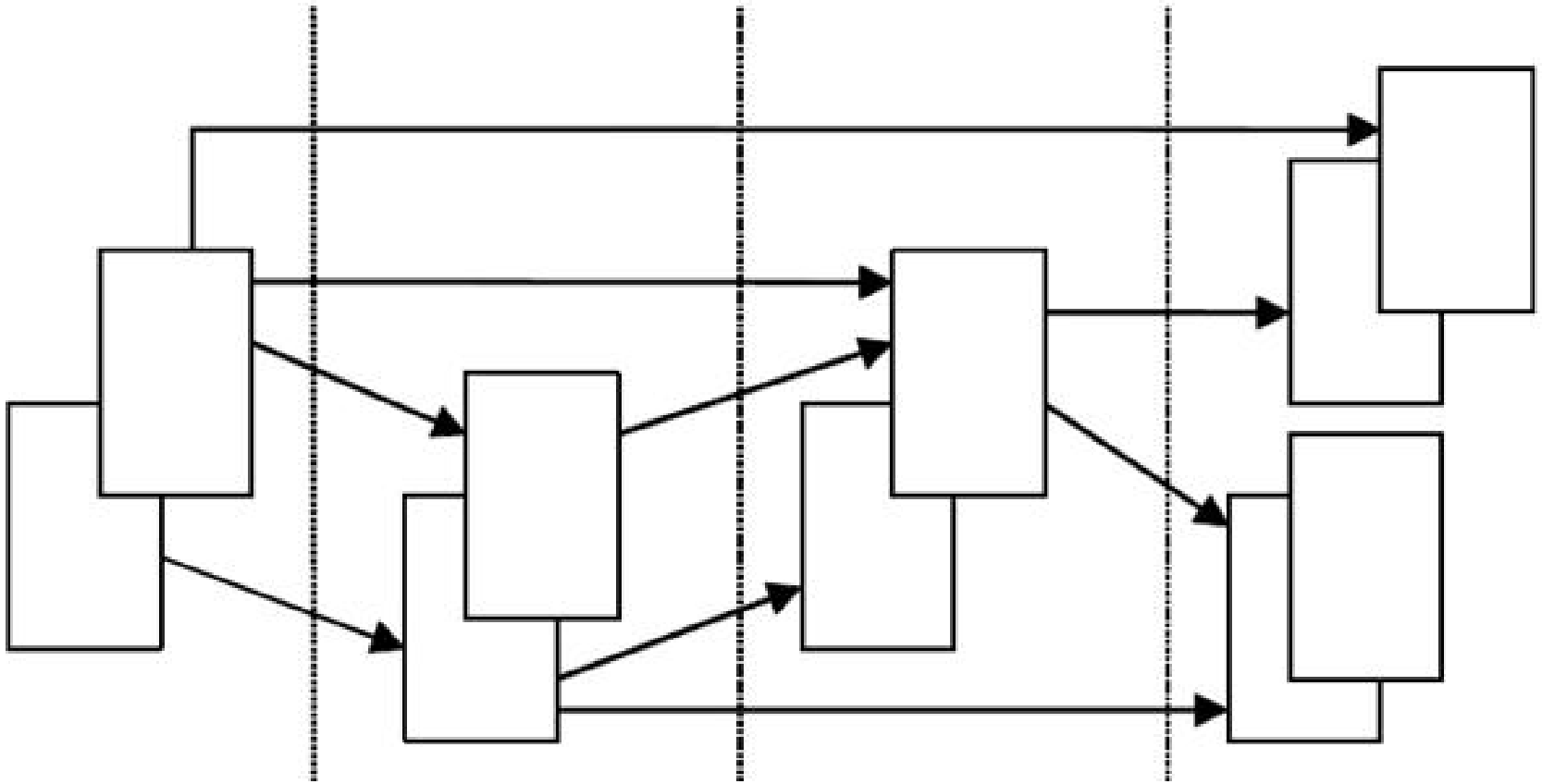
付款

物流

目 录

- 基本概念
- 团队基因
- 领域要素
- 举个例子
- 总结思考

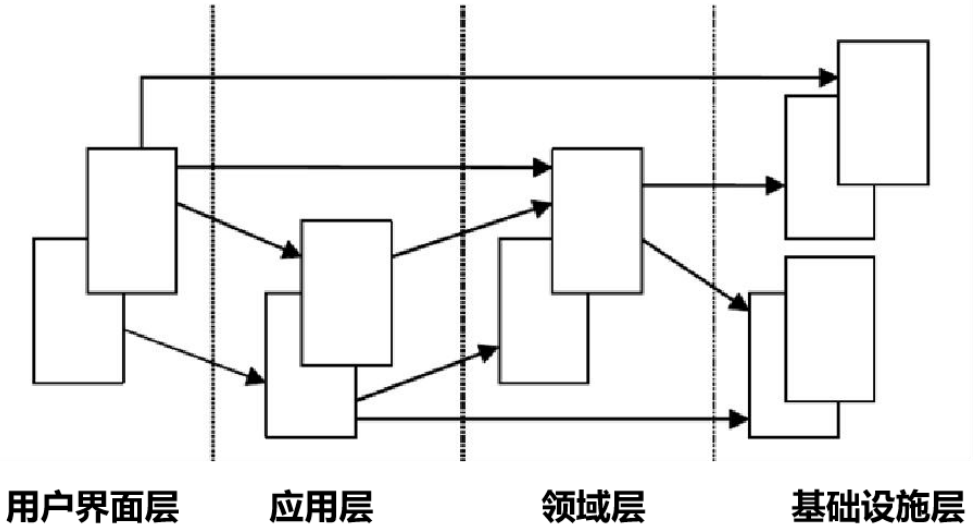
领域驱动设计的经典分层架构



用户界面层　　应用层　　领域层　　基础设施层

用户界面/展现层

负责向用户展现信息以及解释用户命令

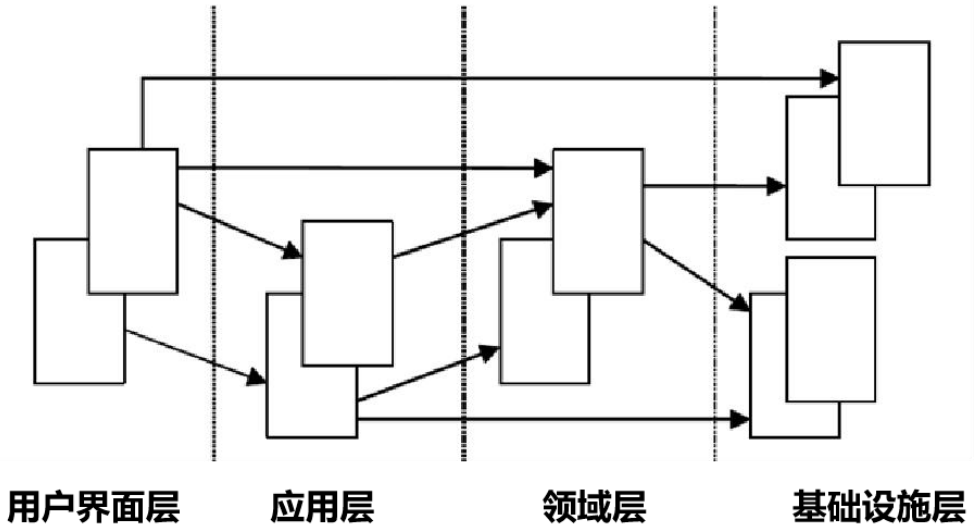


应用层

主要用于协调不同领域对象之间的动作或领域模型与基础结构层组件之间的工作，以完成一个特定的、明确的系统任务。

很薄的一层，不包含业务逻辑，不保留业务对象的状态。

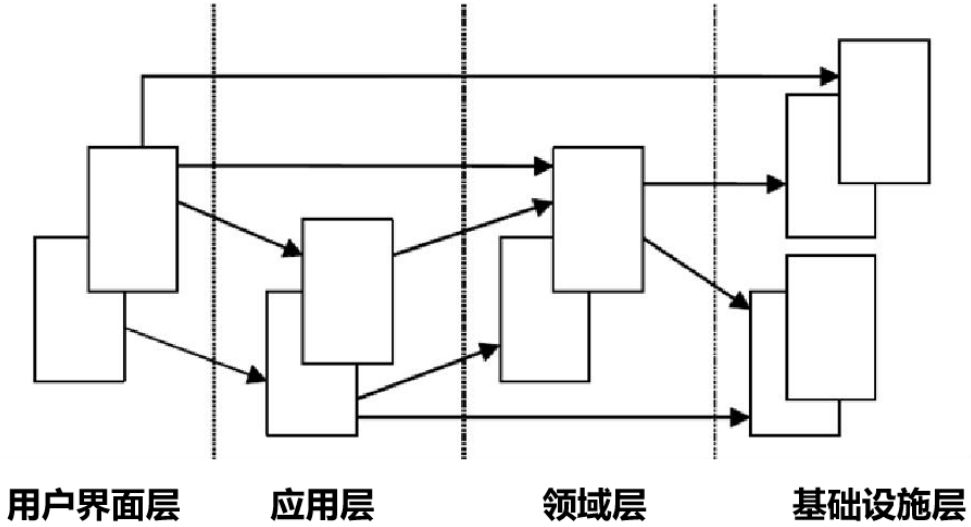
保留应用任务的进度状态。



领域层

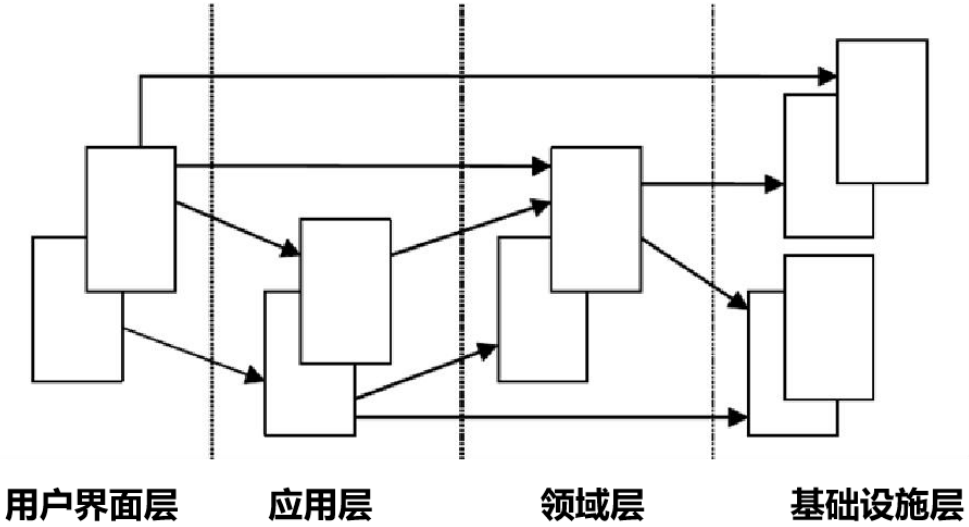
包含关于领域的信息。这是业务软件的核心所在。

保留业务对象的状态。业务对象和它们状态的持久化被委托给基础设施层。



基础设施层

作为其他层的支撑库
提供了层间的通信
实现业务对象的持久化
包含对用户界面层的支持库等



领域层概念

- 实体
- 值对象
- 领域服务
- 聚合
- 资源库
- 工厂
- 上下文

实 体

- 具有生命周期
- 有唯一标识
- 通过Id判断相等性
- 增删改查/持久化
- 可变
- 比如Order/Car

VS

值对象

- 起描述性作用
- 无唯一标识
- 通过属性判断相等性
- 实现Equals()方法
- 即时创建/用完即扔
- 不可变(Immutable)
- 比如Address/Color

领域服务

- 服务执行的操作代表了一个领域概念，这个领域概念无法自然地隶属于一个实体或者值对象。
- 被执行的操作涉及到领域中的其他的对象。
- 操作是无状态的。
- 比如，转账

应用服务

- 语言级别API
- 与UI/通信协议无关
- API与业务用例一一对应
- 很薄的一层
- 起协调代理作用
- 本身并不包含业务逻辑
- 事务边界

领域服务 VS 应用服务

不论是应用服务还是领域服务，通常都是建立在领域的实体对象和值对象之上，以便提供与这些对象直接相关的服务。

决定一个服务应该归属于哪一层是很困难的。

如果所执行的操作概念上属于应用层，那么服务就应该放到这个层。

如果操作是关于领域对象的，而且确实是与领域有关的、为领域的需要服务，那么它就应该属于领域层。

聚合

- 定义一组关联的对象，以及对象和关系之间的边界，作为一个数据更改的单元处理。
- 只有一个聚合根（实体对象）。
- 聚合根可以引用其他聚合根，聚合内的对象可以引用聚合内的另一个对象，但是聚合边界外的任何对象不能绕过聚合根对象访问聚合内的对象。

聚合根

- 对外边界
- 其内部的所有业务操作都必须经过聚合根
- 一次业务用例对应一次事务对应一个聚合根
- 聚合根之间的数据一致性通过最终一致性完成

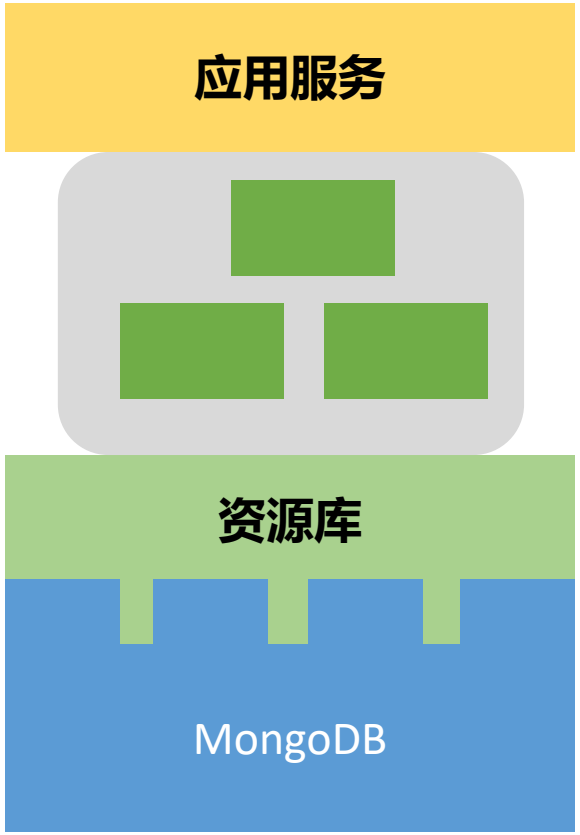
```
//Good
order.removeItem(item);


//Bad
List<Item> items = order.getItems();
items.remove(item);
```

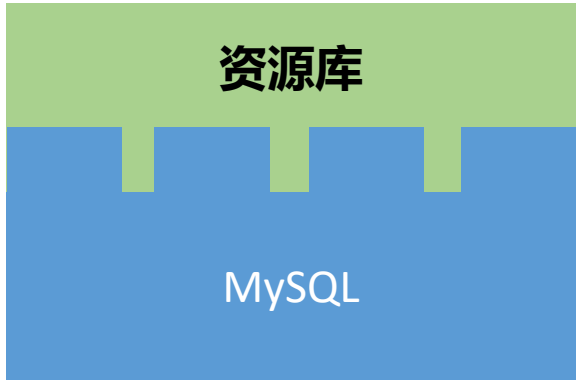
应用服务直接调用聚合根

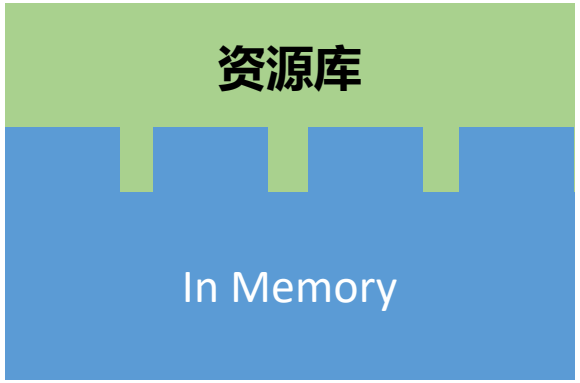
```
public class OrderApplicationService {
    @Transactional
    public void changeAddress(String orderId, String newAddress) {
        Order order = orderRepository.byId(orderId);
        order.changeAddress(newAddress);
        orderRepository.save(order);
    }
}
```

资源库

- 封装所有获取对象引用所需的逻辑
- 保存某些对象的引用



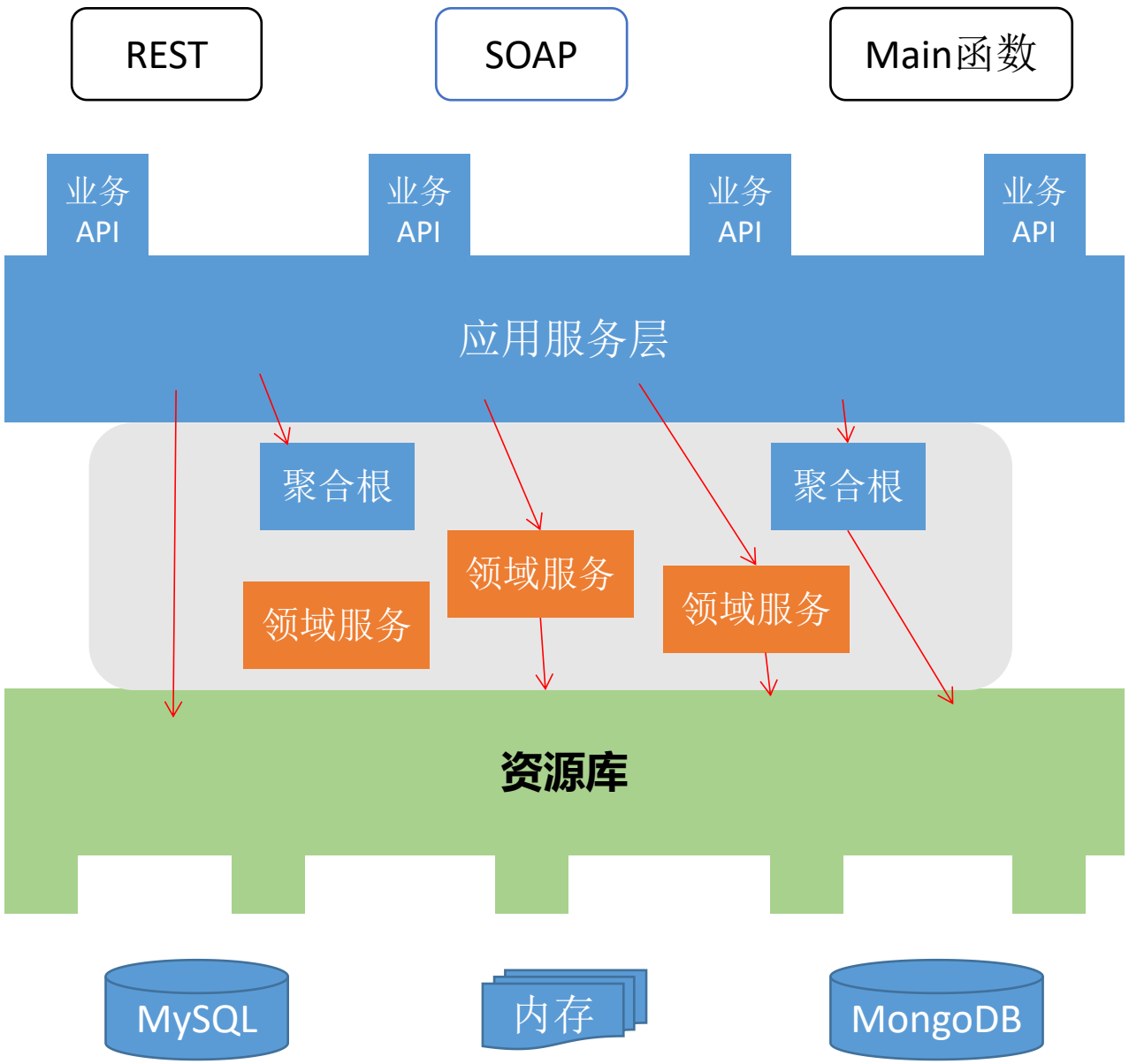




工厂

- 帮助封装复杂的对象创建过程

整体图


REST
SOAP
Main函数
业务
API
业务
API
业务
API
业务
API
应用服务层
聚合根
聚合根
领域服务
领域服务
领域服务
资源库
MySQL
内存
MongoDB

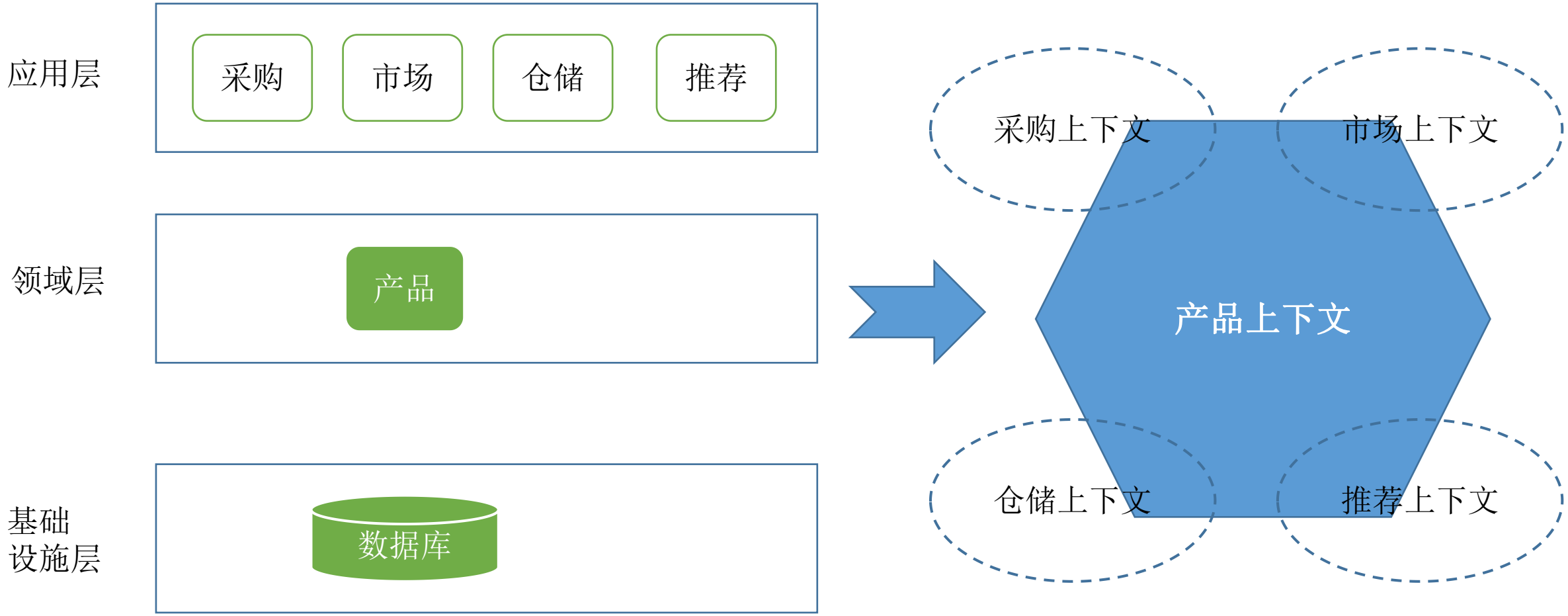
限界上下文(bounded context)

” 对一个大型系统，领域模型的完全统一将是不可行的或者不划算的。 “ —— Eric Evans

DDD的构建块不能盲目地应用在一个无限大的领域模型上，一个无限大的领域模型也无助于开发出优质的软件，限界上下文是分解领域模型的关键。

“限界上下文是一个单词或是一个句子所出现的环境，这个环境会反过来影响它们的含义”

限界上下文确定了了业务边界



一个领域概念在一个限界上下文中不应有二义性

选购

```
public class Order {
    private Item
    private
    ad
    re
}
```

付款

```
public class Order {
    private d
    priva
    n
    issueInvoice();
```

物流

```
public class Order {
    private String
    private St
    priva
    n
    changeAddress();
}
```

界限上下文

界限上下文

界限上下文

目 录

- 基本概念
- 团队基因
- 领域要素
- 举个例子
- 总结思考

住房贷款处理系统

- 业务用例是批准住房贷款（抵押）的资金申请。将贷款申请提交给抵押放贷公司的时候，首先要通过承保过程，承保人在这一过程中根据客户的收入详情、信用历史记录和其它因素来决定批准还是拒绝贷款请求。如果贷款申请获得承保组的批准，就进入贷款审批程序的结清和融资步骤。
- 贷款处理系统中的融资模块自动给贷款人支付资金。通常，融资过程从抵押放贷公司（通常是银行）将贷款包递交给产权公司开始。接着产权公司评估贷款包，并与房产买卖双方一起确定结清贷款的时间。贷款人和卖方与结算中介在产权公司会面、签署书面协议，来转移房产产权。

领域对象

实体：

- Loan
- Borrower
- UnderwritingDecision
- FundingRequest

值对象：

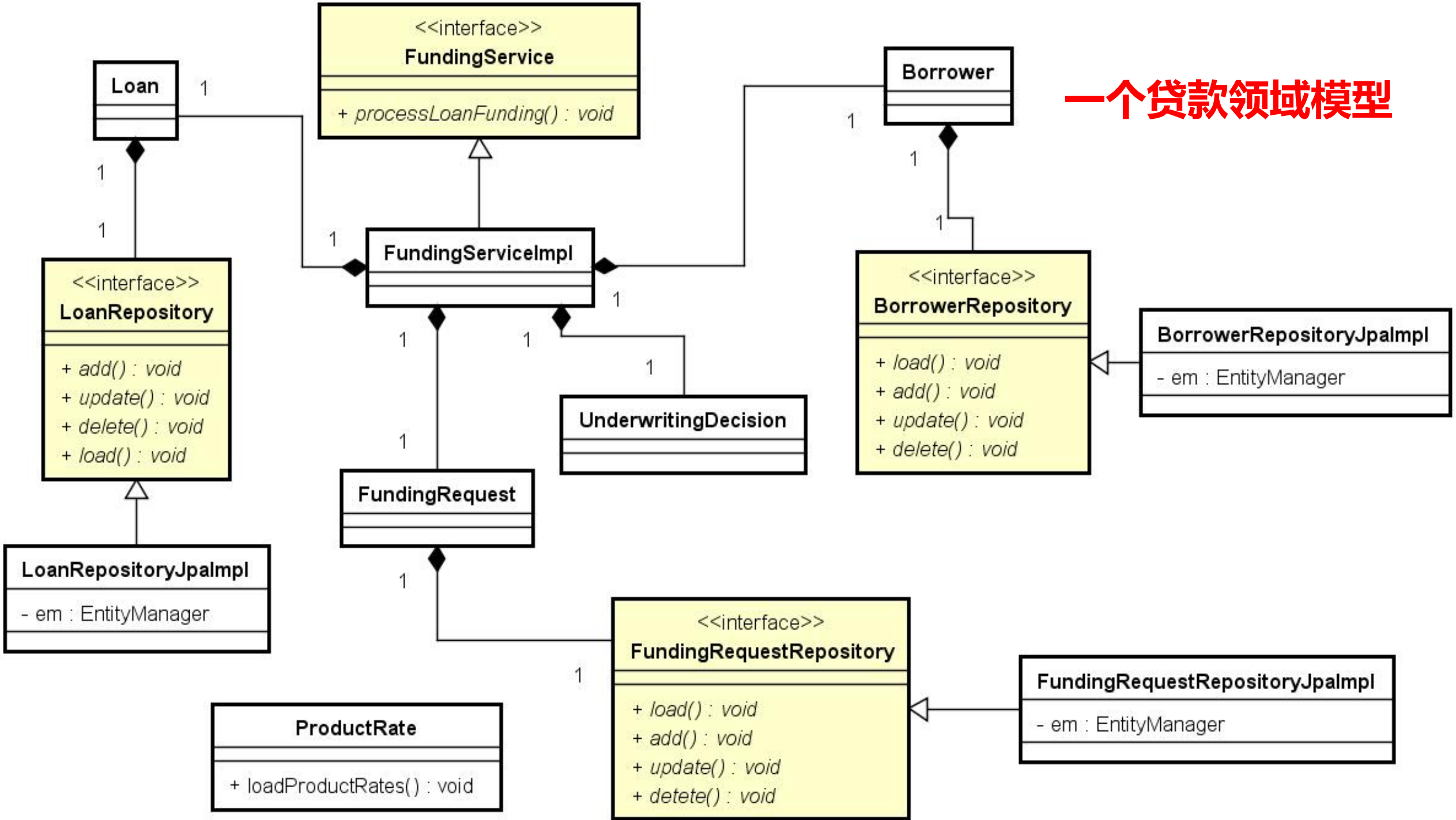
- ProductRate
- State

服务：

- FundingService

资源库：

- LoanRepository
- BorrowerRepository
- FundingRepository


一个贷款领域模型
<<interface>>
FundingService
+ processLoanFunding() : void
Loan
Borrower
<<interface>>
LoanRepository
+ add() : void
+ update() : void
+ delete() : void
+ load() : void
FundingServiceImpl
<<interface>>
BorrowerRepository
+ load() : void
+ add() : void
+ update() : void
+ delete() : void
BorrowerRepositoryJpaImpl
- em : EntityManager
UnderwritingDecision
FundingRequest
LoanRepositoryJpaImpl
- em : EntityManager
<<interface>>
FundingRequestRepository
+ load() : void
+ add() : void
+ update() : void
+ detete() : void
FundingRequestRepositoryJpaImpl
- em : EntityManager
ProductRate
+ loadProductRates() : void
1

目 录

- 基本概念
- 团队基因
- 领域要素
- 举个例子
- 总结思考

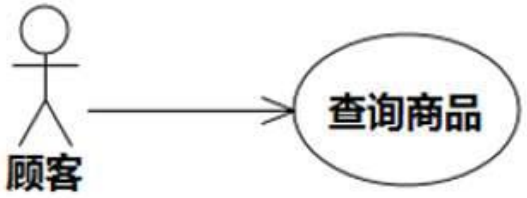
总结

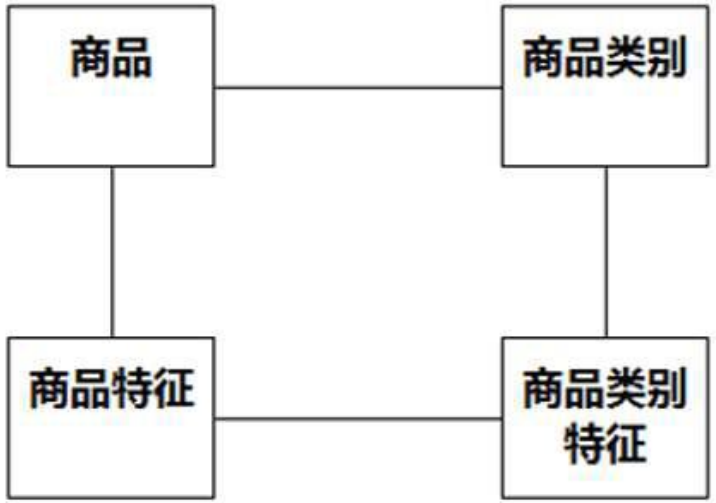
- **领域模型**是软件中最有价值和最具竞争力的部分；设计足够精良且符合业务需求的领域模型能够更快速地响应需求变化。
- **领域模型是排除了人之外的客观世界模型**，虽然领域模型包含人所扮演的参与者角色，但是一般情况下不要让参与者角色在领域模型中占据主要位置。
- **领域模型是与“谁用和怎样用”无关的客观模型**。领域建模是建立虚拟模型让我们现实的人使用，而不是建立虚拟空间，去模仿现实。

总结

- 捕捉“用户心中的模型”，不能等同于“以用户为核心设计领域模型”
- ——不能以'用户为中心'作为出发点去思考问题，不能老是想着**用户会对系统做什么**；
- ——应该从一个客观的角度，根据用户需求挖掘出领域内的相关事物，以这些**事物的本质关联**及其变化规律为出发点去思考问题。

总结：人在领域范围之外



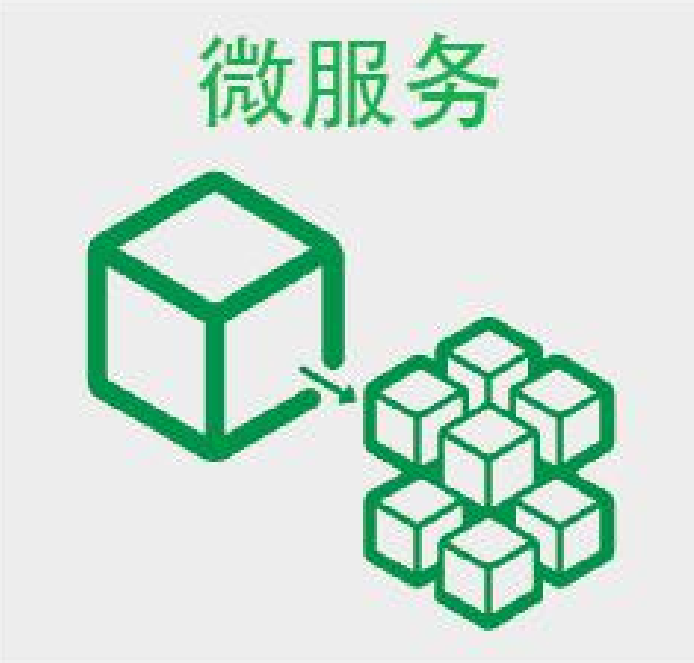


总结：一个货物运输系统

以货物为中心，把客户看成是货物在某个场景中可能会涉及到的关联角色，如货物会涉及到托运人、收货人、付款人；货物有一个确定的目标，货物会经过一系列的运输动作到达目的地。

- 一个货物涉及多个用户（客户，如托运人、收货人、付款人），每个用户承担不同的角色；
- 货物的运送目标已指定，即货物有一个运送目标；
- 由一系列满足规格的运输动作来完成运输目标；
- 如果是以人为中心的话，就变成了：托运人托运货物，收货人收货物，付款人付款，等等；

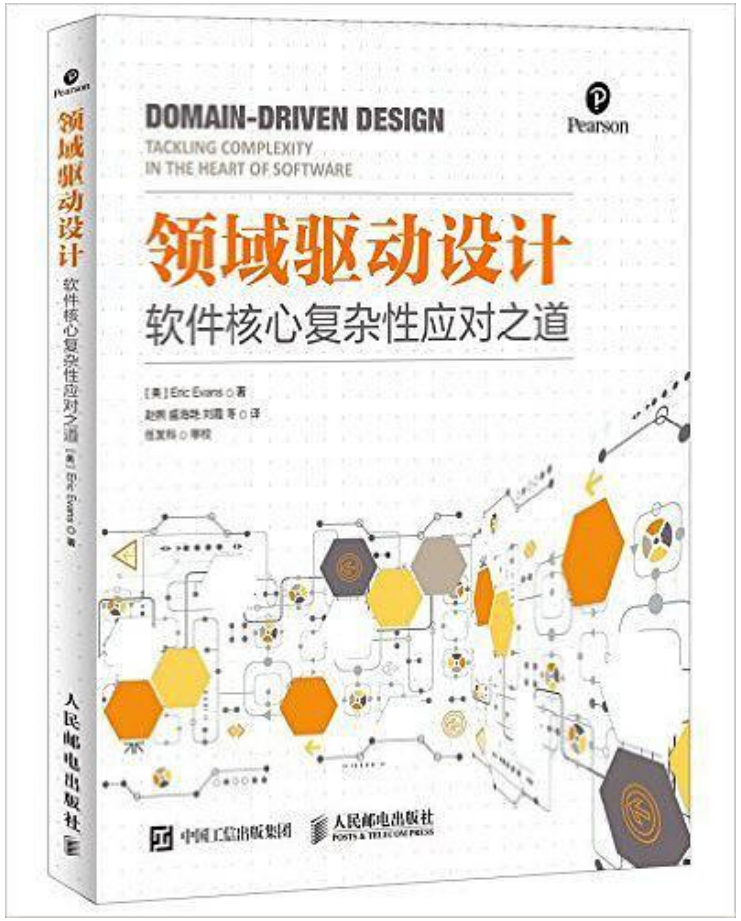
总结：微服务与DDD



总结：产品的类型

工具类 vs 业务类

进一步学习


DOMAIN-DRIVEN DESIGN
TACKLING COMPLEXITY IN THE HEART OF SOFTWARE
Pearson
领域驱动设计
软件核心复杂性应对之道
人民邮电出版社

TURING 图灵程序设计丛书
Addison Wesley
Applying Domain-Driven Design and Patterns
领域驱动设计与模式实战
[瑞典] Jimmy Nilsson 著
赵俐 马燕新 等译
Martin Fowler 和Eric Evans 两位大师联袂推荐
.NET开发人员必读之作
《企业应用架构模式》与《领域驱动设计》两大名著精髓的实战演练
教你穿越业务层、数据层和UI层之间重重障碍，打通任督二脉
人民邮电出版社
POSTS & TELECOM PRESS

DDD Community
Broadview
PEARSON
实现
IMPLEMENTING DOMAIN-DRIVEN DESIGN
领域驱动设计
Vaughn Vernon 著
滕云 译
张逸 审
· 涵盖DDD各个方面
· 大量示例代码
· 案例研究贯穿全书
· 理论和实践紧密结合
· 程序员进阶佳作
DDD之父Eric Evans作序
電子工業出版社